

Спецификация и план разработки

Тема проекта: Визуализация алгоритма Флойда-Уоршелла

1. Общие положения

Проект представляет собой приложение с графическим интерфейсом для визуализации алгоритма нахождения кратчайших путей между всеми парами вершин во взвешенном ориентированном графе - алгоритма Флойда-Уоршелла.

Приложение позволит пользователю наглядно увидеть, как динамическое программирование последовательно улучшает оценки длин путей при добавлении промежуточных вершин.

2. Функциональные требования

Основные возможности пользователя

Добавление вершин: Создание новой вершины на холсте по двойному щелчку мыши. Новая вершина появляется в точке щелчка, ей автоматически присваивается уникальный идентификатор (буква или номер). Вершина сразу становится доступной для редактирования.

Редактирование ребер: Создание ориентированного ребра (дуги) между двумя вершинами осуществляется следующим образом: пользователь выбирает первую вершину (начало дуги) щелчком мыши, затем выбирает вторую вершину (конец дуги) щелчком (или через контекстное меню: правый клик на вершине → пункт "Добавить ребро из...", затем выбор конечной вершины). После выбора пары вершин появляется диалоговое окно для задания веса ребра (целое или вещественное число). Вес может быть положительным, отрицательным или нулевым.

Удаление элементов: Удаление выбранной вершины или ребра выполняется клавишей "Delete" (или через контекстное меню → пункт "Удалить"). При удалении вершины автоматически удаляются все инцидентные ей рёбра.

Перемещение вершин: Перетаскивание вершин по холсту с зажатой левой кнопкой мыши для удобного расположения графа.

Ввод/Вывод данных:

Ручной ввод: Задание матрицы смежности графа в табличном виде прямо в интерфейсе приложения (отдельная вкладка или диалоговое окно с редактируемой таблицей $n \times n$, где в ячейки вводятся числа или INF).

Загрузка из файла: Чтение графа из текстового файла (формат: первая строка - количество вершин n ; следующие n строк - координаты каждой вершины (X, Y) в виде двух целых чисел, разделённых пробелом; затем матрица смежности $n \times n$, где INF обозначает отсутствие ребра).

Сохранение в файл: Экспорт текущего состояния графа в тот же текстовый формат.

Управление визуализацией (Алгоритм):

Запуск: Кнопка "Запустить алгоритм" - выполняет алгоритм непрерывно до завершения с задержкой между шагами.

Пошаговый режим: Кнопки "Шаг вперед" / "Шаг назад" - выполняют переход на одну итерацию внутреннего цикла с обновлением отображения.

Индикация: На каждом шаге алгоритма отображаются три счётчика:

- **k** (внешний цикл) - текущая промежуточная вершина (выделяется на холсте, например, обводкой красного цвета и подсветкой метки);
- **i** (средний цикл) - вершина-источник (выделяется синим цветом);
- **j** (внутренний цикл) - вершина-назначение (выделяется зелёным цветом).

Все три счётчика отображаются в панели состояния или в логе выполнения.

Обновление ребер: При улучшении кратчайшего пути (условие $D[i][k] + D[k][j] < D[i][j]$) ребро (i, j) на холсте временно выделяется: меняет цвет на ярко-зелёный и увеличивает толщину на 2 пикселя. Выделение сохраняется до следующего шага, после чего ребро возвращается к обычному стилю. В матрице соответствующий элемент обновляется и подсвечивается жёлтым цветом. Если ребро отсутствовало (было INF), оно рисуется пунктирной линией после обновления.

Взаимодействие с результатом:

Матрица результатов: Отображение текущей и финальной матрицы кратчайших расстояний в таблице справа от холста. Значения, которые были обновлены на последнем шаге, подсвечиваются жёлтым фоном.

Просмотр пути: Возможность кликнуть на ячейку финальной матрицы (между вершинами i и j) и увидеть на холсте путь (цепочку ребер), соответствующий этому кратчайшему пути (восстановление по матрице next). Путь выделяется жирными рёбрами оранжевого цвета, вершины пути обводятся оранжевым.

3. Проектирование пользовательского интерфейса

Ниже представлен макет главного окна приложения.

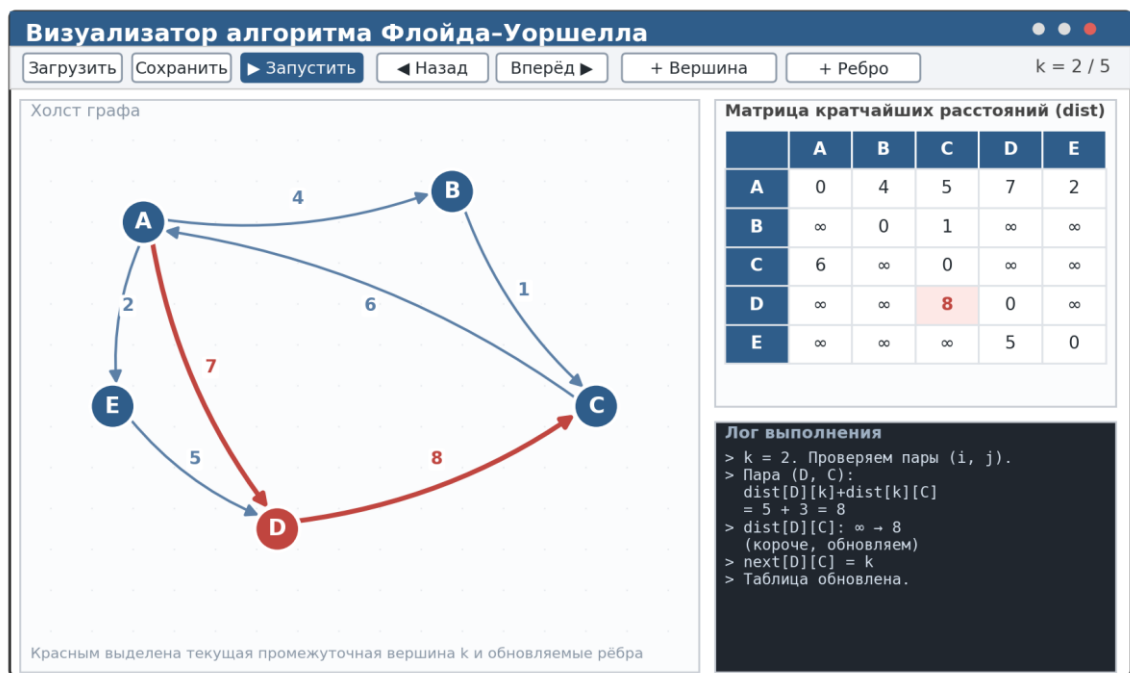


Рис. 1. Макет графического интерфейса.

Пояснение: Холст занимает левую часть окна. Правая панель разделена на область матрицы (таблица) и область лога/текстовых пояснений.

4. Процесс визуализации

При нажатии кнопки "Шаг вперед" или в режиме автозапуска выполняются следующие действия:

Текущие переменные циклов отображаются в панели состояния:
"Итерация $k = 2$, $i = 1$, $j = 3$ ".

Текущая промежуточная вершина k выделяется на холсте (обводится красным цветом). Вершина i выделяется синим, вершина j - зелёным.

В логе появляется сообщение:
"Проверяем $D[1][2] + D[2][3] = 3 + 1 = 4$? $D[1][3] = 5$ ".

Визуально пробегаются пары (i, j) - на каждом шаге соответствующие вершины подсвечиваются.

Если $D[i][k] + D[k][j] < D[i][j]$, то:

- на холсте ребро (i, j) меняет цвет на ярко-зелёный и увеличивается толщина;
- в логе: **"Найден более короткий путь из A в C: $3 + 1 = 4 < 5$ ";**
- в матрице значение обновляется, ячейка подсвечивается жёлтым.

Если неравенство не выполнено ($D[i][k] + D[k][j] \geq D[i][j]$), то:

- в логе выводится: **"Улучшение не требуется: $D[1][2] + D[2][3] = 4 \geq D[1][3] = 4$ "** (или аналогичное сообщение с конкретными значениями);
- ребро и ячейка матрицы остаются без изменений, никакая подсветка не применяется.

Обновление матрицы: Таблица справа перерисовывается с новыми значениями (изменённые ячейки подсвечиваются).

Окончание: Алгоритм завершается, когда $k == n$. В логе: **"Алгоритм завершен. Найдены все кратчайшие пути."** Все счётчики сбрасываются.

5. План разработки (Итерации)

Итерация 0: Спецификация и план разработки, прототип (Дедлайн: 10 июля)

[Общее]: Создать репозиторий.

[Гриценко К.В. (Модель/Логика)]: Создать необходимые классы. Реализовать алгоритм Флойда-Уоршелла на массивах (без GUI, с вызовом из main для теста).

[Кузьмин Г.И. (Интерфейс/View)]: Разработать макет окна на Swing. Сделать холст (JPanel + Graphics2D) для отрисовки графа (круги, стрелки).

[Харченко Я.К. (Контроллер/Ввод)]: Сделать кнопку "Загрузить". Реализовать кнопки "Шаг вперед", "Шаг назад" (стек состояний).

Результат: Запускаемое окно с пустым холстом, кнопками. Алгоритм работает, но результаты выводятся в консоль (или просто не шагово).

План тестирования: 1) unit-тесты алгоритма Флойда-Уоршелла (JUnit 5) - корректность на графах без циклов, обнаружение отрицательного цикла, граничный случай графа из одной вершины; 2) ручные сценарии проверки GUI - загрузка графа из файла, добавление/удаление вершины и ребра, пошаговое выполнение вперед/назад, сверка итоговой матрицы `dist` с эталонным расчётом; 3) проверка сохранения графа в файл и повторной загрузки (round-trip).

Итерация 1: Базовая функциональность (Дедлайн: 13 июля)

[Гриценко К.В.]: Интегрировать алгоритм с визуализацией. Сделать метод `step()`, который выполняет одну итерацию `k` и возвращает список изменений (какие ребра обновились).

[Кузьмин Г.И.]: Реализовать динамическую отрисовку: отображение весов ребер, подсветку текущей вершины `k`, анимацию изменения цвета ребер при обновлении `dist`.

[Харченко Я.К.]: Реализовать загрузку графа из файла и парсинг в модель. Добавить таблицу для отображения матрицы `dist`.

Результат: Приложение позволяет загрузить граф, пошагово пройти алгоритм, видя изменения на холсте и в таблице.

Итерация 2: Улучшения (Дедлайн: 15 июля)

[Гриценко К.В.]: Реализовать восстановление пути по матрице `next`. Написать функцию, возвращающую список вершин для пути $i \rightarrow j$.

[Кузьмин Г.И.]: Реализовать "Drag-and-Drop" для вершин. Добавить контекстное меню (правый клик) для добавления/удаления ребер.

[Харченко Я.К.]: Связать клик по ячейке в матрице с отображением пути на холсте. Реализовать "Сохранение графа в файл".

Результат: Полноценное интерактивное приложение. Можно рисовать граф мышкой, запускать алгоритм и просматривать конкретные кратчайшие пути.

Итерация 3: Полировка и тесты (Дедлайн: 17 июля)

[Все]: Проверка на краевые случаи: граф с 1 вершиной, отрицательные веса (проверка на циклы).

[Гриценко К.В.]: Написать unit-тесты для алгоритма (JUnit 5): корректность на графах без циклов, с отрицательными весами без отрицательных циклов, определение отрицательного цикла, граф из 1 вершины.

[Кузьмин Г.И.]: Улучшить UI (иконки, тултипы, цветовая схема). Добавить полосу прогресса.

[Харченко Я.К.]: Собрать финальный .jar файл через Maven (запуск `java -jar`, совместимо с Java 21, без внешних зависимостей). Проверить запуск вне IDE на чистой машине. Написать разделы отчёта по своей части.

Результат: Готовый продукт, защищенный от ошибок, с красивым интерфейсом.

Роли:

Backend Developer (Гриценко К.В.):

Ответственность: Архитектура модели (Model), логика алгоритма, работа с файлами (ввод/вывод матриц).

UI/UX Developer (Кузьмин Г.И.):

Ответственность: Все, что касается графического интерфейса (Swing, Graphics2D), отрисовка графа, анимация, дизайн панелей.

Frontend/Controller Developer (Харченко Я.К.):

Ответственность: Связь между моделью и представлением (MVC), обработка событий мыши/клавиатуры, логика кнопок, управление состоянием (шаги), сборка проекта.

6. Журнал изменений требований

09.07.2026 получены следующие замечания (исправления внесены в отчет):

0) Про используемые технологии — это войдёт в другой раздел отчёта.

1) Управление графом: описать подробнее каждое действие. (Например, создание дуги "через контекстное меню или панель инструментов" — это как?)

2) "Индикация: Подсветка текущей обрабатываемой промежуточной вершины k на каждом шаге."

Показывать и другие две переменные. Всего должно быть 3 счётчика, связанных с выполнением "крупных", "средних" и "мелких" итераций.

3) "Обновление ребер: Визуальное изменение толщины/цвета ребер при улучшении кратчайшего пути (обновление матрицы $dist$)."

Напишите подробнее, когда и каким цветом что выделяется. В том числе в матрице.

4) "Если $D[i][k] + D[k][j] < D[i][j]$, то на холсте ребро (i, j) меняет цвет/толщину, и в матрице обновляется значение."

А если неравенство не выполнено? В логге нужно что-то вывести и в этом случае.

5) Упоминания очков из спецификации и отчёта уберите.